

05. Referencia técnica

PDF Reader v0.3.2 | Análisis 2026-09-28 | Commit del release 0638d05

Contenido

- Estado global y constantes
- Funciones de `src/app.js`
- Persistencia y utilidades
- Adaptadores nativos
- Configuración, rutas y códigos de salida
- Riesgos de modificación

Estado global y constantes

`src/app.js` usa `APP_VERSION = '0.3.2'`, un mapa `els` de elementos DOM y `state` como estado mutable de sesión. Sus campos incluyen documento, página, zoom, rotación y secuencias independientes para carga, render, búsqueda y miniaturas; estas últimas invalidan trabajo obsoleto al cambiar de documento o cerrar un panel.

`src/history-store.js` define `DB_NAME=pdf-reader-local`, `DB_VERSION=1`, `STORE=documents` y `MAX_DOCUMENTS=8`. Android limita aperturas externas a `MAX_PDF_BYTES=128 MiB`.

Funciones de `src/app.js`

Símbolo	Firma/retorno	Propósito y efectos
<code>setLoading</code>	<code>(on, label?) -> void</code>	Actualiza indicador de carga
<code>showToast</code>	<code>(message) -> void</code>	Muestra aviso temporal y reinicia timer
<code>setError / clearError</code>	<code>(message?) -> void</code>	Cambia vista y estado de error
<code>closePanels, setView</code>	<code>(view?) -> void</code>	Navegación Lector/Historial/About y hash
<code>applyTheme, toggleTheme</code>	<code>(theme?) -> void</code>	DOM, <code>localStorage</code> y color de tema
<code>captureReadingAnchor</code>	<code>(clientX?, clientY?) -> Anchor o null</code>	Convierte un punto visible en razón del canvas
<code>restoreReadingAnchor</code>	<code>(Anchor?) -> void</code>	Ajusta scroll tras redimensionar canvas
<code>updateUi</code>	<code>() -> void</code>	Sincroniza controles con <code>state</code>
<code>saveReadingState</code>	<code>() -> void</code>	Escribe progreso en <code>localStorage/IndexedDB</code>
<code>restoreReadingState</code>	<code>() -> void</code>	Valida y restaura progreso local
<code>refreshHistory, renderHistory</code>	<code>() -> Promise<void>/void</code>	Consulta y crea tarjetas seguras con DOM APIs
<code>deleteHistoryItem, clearAllHistory</code>	<code>(id?) -> Promise<void></code>	Elimina persistencia tras acción explícita
<code>choosePdf, openDesktopPath, loadFile</code>	<code>(file/path?) -> Promise<void></code>	Adquiere un PDF según plataforma
<code>openFromHistory</code>	<code>(id) -> Promise<void></code>	Recupera bytes o ruta guardada
<code>loadPdfData</code>	<code>(raw, fileMeta, options?) -> Promise<void></code>	Destruye documento anterior, abre, persiste y renderiza
<code>friendlyPdfError</code>	<code>(error) -> string</code>	Traduce contraseña/formato/error genérico
<code>effectiveScale</code>	<code>(page) -> Promise<number></code>	Calcula ancho, página o zoom manual
<code>renderPage</code>	<code>({anchor?}) -> Promise<void></code>	Renderiza canvas con <code>DPR <= 2</code> y cancela tarea previa
<code>goPage, zoom, setFit, rotate</code>	<code>(...) -> Promise<void></code>	Cambian vista y solicitan render

Símbolo	Firma/retorno	Propósito y efectos
buildThumbnails	() -> Promise<void>	Genera botones para todas las páginas y canvases hasta 120
pageText, searchDocument	(page/query) -> Promise<...>	Extrae/cacha texto y crea resultados escapados
toggleSidebar, toggleSearch, toggleFullscreen	() -> void/Promise	Controla paneles y Fullscreen API
shareReading	(file?, page?, total?) -> Promise<void>	Capacitor Share, Web Share o WhatsApp Web
appPlatform, offerDefaultReader, configureDefaultReader	(...)	Detecta superficie y abre configuración opcional
decodeBase64, openPendingAndroidPdf	(value?) -> Uint8Array/Promise	Convierte payload Android y lo abre
startTouch, moveTouch, endTouch	(TouchEvent) -> void	Pinza, swipe y doble toque
boot	() -> Promise<void>	Tema, vista, historial y aperturas iniciales

Los handlers registrados al final conectan botones, teclado, resize, drag/drop, gestos, hash e intent Android. Las excepciones de render se muestran; varias fallas de persistencia se registran y degradan sin detener lectura.

Persistencia y utilidades

history-store.js exporta listHistory, getHistoryDocument, saveHistoryDocument, updateHistoryDocument, removeHistoryDocument y clearHistory; todas abren/cerran la base y esperan la transacción. requestResult, transactionDone y database son internas.

utils.js exporta clamp, readingAnchorDelta, formatBytes, documentKey, historyId, touchDistance, buildShareText, buildSnippet, escapeHtml y highlightSnippet. Son funciones puras; highlightSnippet depende del escape previo para que el uso posterior de innerHTML no interprete contenido del PDF.

Adaptadores nativos

Interfaz	Operación	Resultado/error
desktopPdf.pick()	Diálogo .pdf	{canceled} o metadatos+Buffer
desktopPdf.openPath(path)	Lee ruta validada por extensión	objeto o {error}
desktopPdf.startup()	Ruta recibida al iniciar	{path}
desktopPdf.openDefaultApps()	Abre ms-settings:defaultapps	promesa IPC
desktopPdf.onExternalOpen(cb)	Suscribe segunda apertura	callback con ruta
PdfIntent.getPendingPdf()	Lee ACTION_VIEW, máximo 128 MiB	JSONObject Base64 o rechazo
PdfIntent.requestDefault()	Crea PDF temporal y abre resolver	resolución/rechazo

Configuración, rutas y códigos de salida

- PORT: opcional, 4173 por defecto.
- androidScheme: https; allowMixedContent: false.
- appId/applicationId/namespace: cl.vladimiracunadev.pdfreader.
- Scripts de prerequisites terminan con código 2 cuando falta entrada/configuración; errores normales usan 1.
- No hay endpoints HTTP de negocio, eventos de dominio, variables de entorno de aplicación ni códigos de error públicos.

Riesgos de modificación

Cambios en `app.js` pueden afectar simultáneamente tres plataformas. Cambiar claves de identidad rompe reanudación; subir `DB_VERSION` exige migración; cambiar `appId` o firma Android rompe actualizaciones; modificar nombres de jobs obliga a actualizar la protección de `main`.